# Deadline-First Queueing and Guarded DVFS for EdgePlus: Slack-Aware Server Rules for 6G Edge Task Allocation

Keerat Malhotra, Bhavya Maheshwari, Bhumika Aswal
Department, University
City, Country
email@example.com

Abstract—EdgePlus is a feasibility-first multiagent reinforcement learning scheduler for 6G edge computing: a masked cooperative PPO policy chooses the server for every task. Once a task reaches its server, however, it waits in first-come-first-served order and runs at full CPU speed. We add two rules at each server. The first serves the waiting task with the earliest deadline, drops tasks that can no longer meet their deadline, and makes the feasibility mask estimate waits under the same order. The second lowers the CPU frequency only when no other task is waiting and caps the added service time. We reimplement EdgePlus (without its PSO outer loop) in a discrete-event simulator, train it with three seeds per load, and apply the rules to the same frozen policies. On the CityPulse workload used by EdgePlus at 150,000 tasks, on-time completion rises from 80.5% to 99.7% and URLLC deadline misses fall from 55.4% to 0.07%. At 50,000 tasks, energy per task falls by 19.8% while on-time completion stays at 100.0%. On real bus-mounted sensor data from Delhi (AirDelhi), evaluated without retraining, on-time completion rises from 89.3% to 98.8% at 100,000 tasks and from 73.0% to 96.5% at 150,000 tasks. The cost is a longer overall 95th-percentile latency at high load (148 s to 298 s on CityPulse), because tasks with 120–600 s deadlines are deliberately served later. Ablations show that each partial rule set performs worse than the combination.

Index Terms—6G edge computing, task scheduling, earliest deadline first, DVFS, multiagent reinforcement learning, energy efficiency

## I. Introduction

Edge servers in a 6G smart city must finish sensor-driven tasks before deadlines that differ by two orders of magnitude: an air-quality alert may have a few seconds, while a logging task may have ten minutes. EdgePlus [1] addresses this with a feasibility-first design. A masked cooperative PPO policy with a centralized critic picks a server among those that respect the routing order, slice budgets, resource limits and deadline estimates; a lightweight advisory layer (Nash sharing, PID and leader election) biases the logits; a constricted PSO outer loop tunes the training; and a two-tier ledger feeds reputation and debt signals back into the state, reward and masks. In the paper's ablations, the feasibility masks account for most of the gain.

EdgePlus decides where a task runs, but not how the server handles it. Its queue model is Kingman's single-server approximation [2], which assumes first-come-first-served (FCFS) order; the high/moderate/low priority label is only used to break ties; and compute energy is modelled as $\kappa_s k_i$, independent of CPU frequency. Under heavy load this matters: in our reimplementation at 150,000 tasks, 55% of URLLC tasks (5–15 s deadlines) miss their deadline, mostly because they wait behind mMTC tasks whose deadlines are 120–600 s. EdgePlus defines a URLLC tail-reliability KPI but does not report per-class results, so this effect is not visible in its published figures. A recent survey of edge computing with 6G and cloud computing also lists workload classification and energy-efficient job distribution among the open challenges [3].

We study two server-side rules that use each task's deadline slack:

- Deadline-first queueing with dropping. Each server serves the waiting task with the earliest deadline (EDF) and drops a task once it cannot finish in time even at full speed. The feasibility mask estimates waits under the same order.
- Guarded DVFS. A task runs at a lower CPU frequency only if no other task is waiting for that server; the slow-down uses at most 80% of the task's slack and adds at most 1.5 s.

Both rules are fixed policies, so they can be added to a trained EdgePlus scheduler without retraining. Our contributions are: (i) the two rules and a mask consistent with them; (ii) an open reimplementation of EdgePlus calibrated against the published results; and (iii) an evaluation on the CityPulse workload used by EdgePlus and on the real AirDelhi dataset [4], with per-class metrics, ablations and paired statistics. Code, trained policies and raw results are released with this paper.

## II. Background and Related Work

### A. EdgePlus

EdgePlus [1] models a 100 $\mathrm{km}^2$ city with ten clusters, stationary and mobile edge servers and a cloud. Tasks belong to three slices (URLLC, eMBB, mMTC) with deadlines drawn from $U[5, 15]$ s, $U[30, 120]$ s and $U[120, 600]$ s. The latency of task $i$ on server $s$ is $L_{i,s} = L^{tx} + L^{move} +$

$L^{queue}+L^{proc}$ and its energy is $E_{i,s} = E^{tx}+E^{move}+E^{comp}$ with $E^{comp} = \kappa_s k_i$. The reported results at 150,000 tasks are 82.77% on-time completion, median latency 53.2 s and 95th-percentile latency 147.1 s; without PSO (ablation A2) the on-time rate is 82.0%.

### B. Deadline scheduling and overload

On a single preemptive processor, EDF meets every deadline of a periodic task set whenever its utilization does not exceed one [5], and ordering jobs by due date minimizes the maximum lateness on a single machine [6]. Under overload, however, EDF can make many tasks late in a row, the so-called domino effect, which is why overload handling such as rejecting or dropping hopeless tasks is used in practice [7], [8]. Our ablation tests whether this applies when a learned dispatcher sits in front of the servers.

### C. Slack-based DVFS

Dynamic CPU power grows roughly with the cube of frequency, so the dynamic energy per cycle grows with its square, while static power favors finishing quickly. Real-time DVFS exploits slack to lower frequency without missing deadlines [9], [10]. On a shared edge server, slowing one task also delays tasks that arrive later, which motivates the guard in our rule.

## III. System Model

We keep EdgePlus's model and change only what happens inside a server. Each server $s$ has $c_s$ cores with maximum frequency $f_s$; task $i$ needs $k_i$ cycles, so its service time at full speed is $w_{i,s} = k_i/f_s$. A task arriving at time $t_i$ with relative deadline $d_i$ has absolute deadline $D_i = t_i+d_i$ and is on time if it finishes by $D_i$. Each server has three rule settings:

- queue order: FCFS (EdgePlus), EDF, or fixed class priority (URLLC > eMBB > mMTC);
- late-task rule: keep (EdgePlus) or drop a task when $t + w_{i,s} > D_i$ at the moment it would start;
- frequency: always $f_s$ (EdgePlus), unguarded DVFS, or guarded DVFS.

With a normalized frequency $\phi \in (0, 1]$, we model compute energy as

$$E^{comp}_{i,s}(\phi) = \kappa_s k_i \left(\alpha\phi^2 + \frac{1-\alpha}{\phi}\right), \tag{1}$$

where $\alpha = 0.75$ is the dynamic share at full speed. The first term is dynamic energy; the second is static power over the longer busy time. At $\phi = 1$, (1) equals EdgePlus's $\kappa_s k_i$, so the baseline's energy is unchanged; the energy-optimal level is near $\phi \approx 0.55$.

## IV. Slack-Aware Server Rules

### A. Deadline-first queueing with dropping

When a core becomes free, the server removes the waiting task with the earliest $D_i$. If it cannot finish in time even at full speed, it is dropped, counted as a failure, and the next task is examined (Algorithm 1). Dropping frees capacity that FCFS would spend on tasks that are already lost.

**Algorithm 1** Server rules when a core becomes free at time $t$

```
while queue Q_s not empty do
    i ← arg min_{j∈Q_s} D_j; remove i from Q_s        ▷ EDF
    if t + w_{i,s} > D_i then drop i (failure) and continue
    end if
    σ_i ← D_i − t − w_{i,s}
    if Q_s = ∅ and nothing in flight then Δ ← min(βσ_i, G)
    else Δ ← 0
    end if
    φ ← min{φ ∈ Φ : w_{i,s}/φ − w_{i,s} ≤ Δ}
    run i on the free core for w_{i,s}/φ; return
end while
```

The mask must agree with the queue order. For an arriving task $i$, EdgePlus estimates the wait from the server's load. With EDF, only queued work with an earlier deadline is ahead of the new task:

$$\hat{W}_{i,s} = r^{min}_s + \frac{1}{c_s}\Big(\sum_{j\in Q_s:\, D_j \le D_i} w_{j,s} + w^{flight}_s\Big), \tag{2}$$

where $r^{min}_s$ is the shortest remaining service time when all cores are busy (zero otherwise) and $w^{flight}_s$ is the work already dispatched to $s$ but still in transit. For FCFS the sum runs over the whole queue. Without (2), the mask would keep rejecting urgent tasks that EDF would in fact serve on time.

### B. Guarded DVFS

When task $i$ starts at time $t$, its slack at full speed is $\sigma_i = D_i - t - w_{i,s}$. The rule picks the lowest level $\phi$ in $\{0.5, 0.6, \ldots, 1.0\}$ whose extra service time fits a budget:

$$\frac{w_{i,s}}{\phi} - w_{i,s} \le \Delta_i, \tag{3}$$

$$\Delta_i = \begin{cases} \min(\beta\sigma_i,\, G) & \text{if } Q_s = \emptyset,\ w^{flight}_s = 0, \\ 0 & \text{otherwise,} \end{cases} \tag{4}$$

with $\beta = 0.8$ and $G = 1.5$ s. The unguarded variant uses $\Delta_i = \beta\sigma_i$ regardless of the queue. The guard bounds how much a slowed task can delay an urgent task that arrives while it runs.

## V. Experimental Setup

### A. Reimplementation

We reimplemented EdgePlus in Python as a discrete-event simulator. Each cluster has four stationary nodes with 24 cores each (2.4–3.6 GHz) and one mobile node with 16 cores at 2.4 GHz (15 m/s, 2 km radio range, 2 kJ battery); the cloud has 32 cores at 3.5 GHz and adds 2 s of delay. We calibrated the core count so that a

simple routing rule (lowest routing tier, earliest estimated finish) gives about 100% on-time completion at 50,000 tasks and about 80% at 150,000, matching the paper. Radio rates follow EdgePlus's goodput model with per-slice bandwidths of 20/50/5 MHz, frequency reuse 3, 23 dBm transmit power with a 20 dB beamforming gain and the path loss $128.1 + 37.6 \log_{10} d$ [11]. Slice compute budgets reserve 25/65/35% of a cluster's cores over a 60 s window. The mask applies the routing order, slice budgets, memory, mobile battery, deadline estimates and URLLC reputation quarantine; if no server passes, the server with the earliest estimated finish is used.

The policy follows EdgePlus Sec. III-E: one actor network (two tanh layers of 64 units) shared by all clusters scores each candidate server from task, cluster and server features; the advisory bias (Nash sharing, PID with EdgePlus's gains 1.90/1.20/0.06, leader election) is added to the logits; infeasible servers are masked before the softmax; and a centralized critic (two layers of 128 units) sees the whole city. The reward is EdgePlus's Eq. 7 with reputation and debt shaping (Eq. 15), using class weights 3/1.5/1. We do not implement the PSO outer loop, so our baseline corresponds to EdgePlus's ablation A2. Mobile relocation uses straight-line distance times a 1.3 detour factor instead of CBS–D* Lite, and the ledger is represented only by its reputation and debt signals, since its 0.39 ms validation is off the task path.

### B. Datasets and episodes

CityPulse [12] is the pollution dataset EdgePlus calls Braşov: 449 sites, 7.89 million readings every 5 min from August to October 2014. CityPulse describes the values as generated by a bounded random walk. We use the first 43 days for training and the last 18 for testing. AirDelhi [4] contains real $PM_{2.5}$/$PM_{10}$ readings from low-cost sensors on 13 public buses in Delhi. We use December 2020 (training statistics) and January 2021 (testing), and remove readings taken while a bus was parked, i.e. moved less than 50 m over about 3 min (30.8% of readings; idle buses otherwise put 38% of all readings into three 1 $km^2$ cells), leaving 4.04 million readings.

Following EdgePlus's Algorithm 3, every reading becomes one task. Severity is the largest robust z-score across pollutants plus half of the largest positive rate of change. The 20% most severe tasks of an episode are URLLC, the next 50% eMBB and the remaining 30% mMTC (EdgePlus Table IV), with EdgePlus's deadline ranges. Mean CPU work is 6/12/9 Gcycles per class, scaled with severity and log-normal noise. An episode takes $N$ consecutive readings and compresses their time span into a 600 s arrival window; the loads are $N \in \{50{,}000; 100{,}000; 150{,}000\}$. Clusters come from weighted K-means on each dataset's training positions (density plus exceedance weight) with Delaunay neighbors; the server inventory is the same in both cities.

TABLE I
Our EdgePlus reimplementation against the paper (CityPulse). On-time and cloud share in %, latencies in s.

| Load | Source | On-time | $L_{p50}$ | $L_{p95}$ | Cloud |
|---|---|---|---|---|---|
| 50k | Paper | 100.00 | 3.98 | 10.24 | – |
| | Ours | 100.00 | 3.55 | 7.35 | 0.00 |
| 100k | Paper | 82.50 | – | ≈151 | – |
| | Ours | 99.87 | 4.20 | 11.79 | 0.09 |
| 150k | Paper | 82.77 | 53.23 | 147.09 | 6.1 |
| | Paper, no PSO | 82.0 | 54.8 | 158.9 | 6.6 |
| | Ours, no PSO | 80.52 | 27.95 | 148.10 | 3.78 |
| | Routing rule | 71.99 | 38.39 | 125.01 | 3.79 |

### C. Training and evaluation

We train one EdgePlus policy per load and seed (3 loads × 3 seeds) for 60 PPO updates [13], each on a full training episode, with GAE [14] ($\gamma = 0.95$, $\lambda = 0.9$), clip 0.2, four epochs, minibatch 4096, learning rate $3 \times 10^{-4}$ and entropy coefficient 0.01. Training uses the baseline server rules and takes 25, 45 and 61 CPU-minutes per run at the three loads (6.6 CPU-hours in total) on one core. At 150,000 tasks the training on-time rate rises from 74.6% in the first ten updates to 79.4% in the last ten (stochastic policy), and the trained policy beats the routing rule by 8.5 points in evaluation (80.5% vs 72.0%).

Each frozen policy is evaluated greedily on 10 test episodes per load, under each rule variant, giving 30 paired samples (3 seeds × 10 episodes) per configuration. For Delhi we use the CityPulse-trained policies without retraining. We report means with 95% confidence intervals and compare each variant with the baseline using the Wilcoxon signed-rank test [15] on pairs that share the policy and test episode, with Benjamini–Hochberg correction [16]. Latency percentiles are computed over completed tasks; dropped tasks count as missed deadlines.

## VI. Results

### A. Fidelity of the reimplementation

Table I compares our baseline with the published numbers. At 50,000 tasks both reach 100% on time with similar latencies. At 150,000 tasks our policy reaches 80.5% against 82.0% for EdgePlus without PSO, with almost the same 95th-percentile latency (148 s vs 147–159 s); our median latency and cloud share are lower. The main difference is at 100,000 tasks, where the paper reports 82.5% and we measure 99.9%. CityPulse readings arrive at a flat rate, so in our simulator the medium load is far from saturation; we therefore also report AirDelhi, whose arrivals are bursty.

### B. Main results

Table II and Fig. 1 show the trained EdgePlus policy with and without the rules. On CityPulse at 150,000

tasks, deadline-first queueing with dropping raises on-time completion from 80.5% to 99.6%. URLLC misses fall from 55.4% to 0.07% and eMBB misses from 16.8% to 0.65%, while mMTC misses stay near zero (0.03% to 0.05%); 0.36% of tasks are dropped. Guarded DVFS alone lowers energy per task by 19.6% at 50,000 tasks, 10.1% at 100,000 and 4.4% at 150,000, because servers are rarely idle at high load. Combined, the two rules keep the on-time gains and reduce energy by 19.8%, 10.0% and 5.2%. For the combined rules, every difference in on-time rate and energy against the baseline is significant (BH-adjusted $p < 10^{-3}$, Wilcoxon on 30 pairs). At 50,000 tasks the combined rules lower the on-time rate by 0.002 points (about two tasks in 100,000), which is statistically detectable but negligible.

### C. Real data without retraining

AirDelhi is harder for the baseline: its arrivals are bursty and its clusters are unevenly loaded, so the CityPulse-trained EdgePlus policy completes 98.8%, 89.3% and 73.0% of tasks on time at the three loads. With both rules the same policy reaches 99.8%, 98.8% and 96.5%. URLLC misses fall from 38.6% to 0.97% at 100,000 tasks and from 73.0% to 3.7% at 150,000. Energy per task falls by 11.2%, 5.5% and 7.3%. The rules contain no learned parameters, so they carry over to a new city unchanged, while the policy itself faces a shift in task locations and arrival patterns.

### D. Ablations

Table III isolates each design choice at the heaviest CityPulse load. Dropping late tasks under FCFS raises on-time completion to 91.2% but still misses 40.3% of URLLC tasks and drops 8.8% of all tasks. EDF without dropping reaches 97.4%. We do not observe the domino collapse described for EDF under overload [8], probably because the mask steers new tasks away from servers with long queues, but EDF alone leaves 4.2% of eMBB and 1.4% of mMTC tasks late. Fixed class priority protects URLLC (0.11% misses) but leaves 10.1% of mMTC tasks late. Deadline-first order with dropping gives the best result (99.6%). For DVFS, the unguarded rule saves 28.1% energy at 50,000 tasks but collapses at higher load (66.0% on time at 100,000 and 46.0% at 150,000), because slowed tasks delay urgent tasks that arrive later. The guard keeps the on-time rate within one point of the baseline at every load on both datasets while still saving energy.

### E. Latency trade-off and overhead

Deadline-first order moves waiting time from urgent tasks to tasks with long deadlines (Fig. 2). At 150,000 CityPulse tasks, the 95th-percentile latency of URLLC tasks falls from 92.3 s to 6.2 s and the median latency of all tasks from 28.0 s to 6.0 s, but the 95th percentile of all tasks rises from 148 s to 298 s and that of mMTC tasks from 223 s to 439 s, still within their deadlines (0.05% missed). Because deadlines span 5 to 600 s, a single latency percentile over all classes hides this; we recommend reporting latency per class. On AirDelhi at 150,000 tasks the overall 95th percentile does not change significantly (339 s vs 316 s, $p = 0.74$), and at 100,000 it rises from 110 s to 155 s.

The rules are cheap: a whole simulated episode, including the simulator itself, takes 0.23 ms per task with the baseline rules and 0.30 ms with ours on one CPU core at 150,000 tasks.

## VII. Discussion and Limitations

Our results come from a simulator, and the size of the gains depends on its modeling choices: servers are multi-core nodes simulated event by event, the mask estimates waits from the real backlog rather than EdgePlus's queue formula, PSO, the cryptographic path and CBS–D* Lite are not implemented, and only three training seeds per load were used. We calibrated the simulator against the published on-time rates, but the medium load on CityPulse is much lighter than in the paper (Table I). CityPulse values are generated, so the AirDelhi results are the closer test of real arrival patterns; there we only evaluated policies trained on CityPulse. Dropping assumes that a late result has no value; where late mMTC results are still useful, dropping can be limited to URLLC and eMBB. The energy model assumes a 25% static share and frequency levels down to half speed, and does not include idle power or sleep states, which could add further savings at light load. Finally, the higher overall tail latency is a real cost for applications that care about absolute delay rather than deadlines.

## VIII. Conclusion

Adding two slack-aware rules to EdgePlus's servers, deadline-first queueing with dropping and guarded DVFS, substantially raises on-time completion under heavy load (80.5% to 99.7% on CityPulse, 73.0% to 96.5% on AirDelhi at 150,000 tasks) and lowers energy per task by up to 19.8% at light load, using the same trained policy and no retraining. The ablations show that the details matter: deadline-first order works best together with dropping, deadlines work better than fixed class priorities, and DVFS needs a guard. Future work includes training the policy with the rules in place, training on AirDelhi directly, adding idle and sleep power, and restoring the PSO outer loop.

## Acknowledgment

The authors used an AI coding assistant to help implement the simulator and draft this manuscript. All results were produced by the released code.

TABLE II
EdgePlus with and without the server rules (mean of 30 paired runs). Misses are per class; latencies in seconds over completed tasks; energy in mJ per task.

| Data | Load | Rules | On-time (%) | Missed deadlines (%) URLLC | eMBB | mMTC | $L_{p50}$ | $L_{p95}$ | URLLC $L_{p95}$ | Energy (mJ) |
|---|---|---|---|---|---|---|---|---|---|---|
| CityPulse | 50k | EdgePlus | 100.00 | 0.00 | 0.00 | 0.00 | 3.55 | 7.35 | 4.42 | 27.04 |
| | | + EDF & drop | 100.00 | 0.00 | 0.00 | 0.00 | 3.55 | 7.35 | 4.42 | 27.04 |
| | | + guarded DVFS | 99.99 | 0.04 | 0.00 | 0.00 | 4.73 | 8.24 | 5.48 | 21.76 |
| | | + both | 100.00 | 0.01 | 0.00 | 0.00 | 4.72 | 8.19 | 5.36 | 21.69 |
| | 100k | EdgePlus | 99.87 | 0.66 | 0.00 | 0.00 | 4.20 | 11.79 | 5.48 | 27.61 |
| | | + EDF & drop | 99.99 | 0.04 | 0.00 | 0.00 | 3.82 | 9.72 | 5.02 | 27.59 |
| | | + guarded DVFS | 99.84 | 0.79 | 0.00 | 0.00 | 5.05 | 12.39 | 6.51 | 24.83 |
| | | + both | 99.99 | 0.04 | 0.00 | 0.00 | 4.59 | 10.48 | 5.80 | 24.86 |
| | 150k | EdgePlus | 80.52 | 55.36 | 16.80 | 0.03 | 27.95 | 148.10 | 92.32 | 28.36 |
| | | + EDF & drop | 99.64 | 0.07 | 0.65 | 0.05 | 5.46 | 290.63 | 6.00 | 28.07 |
| | | + guarded DVFS | 79.60 | 57.11 | 17.94 | 0.04 | 29.81 | 155.12 | 95.64 | 27.11 |
| | | + both | 99.65 | 0.07 | 0.64 | 0.05 | 5.99 | 298.37 | 6.24 | 26.88 |
| AirDelhi (no retraining) | 50k | EdgePlus | 98.75 | 5.76 | 0.19 | 0.00 | 4.38 | 25.26 | 10.29 | 24.03 |
| | | + EDF & drop | 99.85 | 0.16 | 0.24 | 0.00 | 3.71 | 26.74 | 5.70 | 23.89 |
| | | + guarded DVFS | 98.58 | 6.54 | 0.22 | 0.00 | 5.42 | 26.21 | 11.03 | 21.61 |
| | | + both | 99.84 | 0.17 | 0.25 | 0.00 | 4.66 | 28.37 | 6.23 | 21.33 |
| | 100k | EdgePlus | 89.26 | 38.58 | 5.98 | 0.11 | 13.28 | 109.55 | 42.06 | 24.60 |
| | | + EDF & drop | 98.74 | 0.95 | 1.55 | 0.99 | 6.39 | 153.86 | 8.26 | 23.96 |
| | | + guarded DVFS | 88.84 | 40.57 | 6.03 | 0.13 | 14.07 | 109.86 | 42.46 | 23.87 |
| | | + both | 98.78 | 0.97 | 1.50 | 0.94 | 7.10 | 154.95 | 8.39 | 23.24 |
| | 150k | EdgePlus | 72.97 | 72.96 | 24.43 | 0.76 | 38.99 | 338.80 | 182.86 | 25.04 |
| | | + EDF & drop | 96.54 | 3.55 | 3.97 | 2.57 | 15.75 | 316.50 | 10.95 | 23.56 |
| | | + guarded DVFS | 72.35 | 74.97 | 24.84 | 0.80 | 39.91 | 341.52 | 174.32 | 24.76 |
| | | + both | 96.47 | 3.68 | 4.04 | 2.59 | 16.86 | 316.13 | 11.00 | 23.21 |

TABLE III
Ablations, CityPulse at 150,000 tasks. On-time and misses in %, $L_{p95}$ in s, energy in mJ per task.

| Rules | On-time | Missed URLLC | eMBB | mMTC | $L_{p95}$ | Energy |
|---|---|---|---|---|---|---|
| EdgePlus (FCFS) | 80.52 | 55.36 | 16.80 | 0.03 | 148.1 | 28.36 |
| FCFS + drop | 91.24 | 40.33 | 1.39 | 0.00 | 86.7 | 26.46 |
| EDF, keep | 97.41 | 0.32 | 4.24 | 1.37 | 298.0 | 28.29 |
| Class priority + drop | 96.49 | 0.11 | 0.89 | 10.14 | 171.0 | 27.35 |
| EDF + drop | 99.64 | 0.07 | 0.65 | 0.05 | 290.6 | 28.07 |
| Unguarded DVFS | 45.96 | 80.11 | 67.55 | 14.16 | 637.0 | 24.48 |
| Guarded DVFS | 79.60 | 57.11 | 17.94 | 0.04 | 155.1 | 27.11 |
| Both (ours) | 99.65 | 0.07 | 0.64 | 0.05 | 298.4 | 26.88 |

## References


[1] B. H. Taher, J. Luo, F. Ghrabat, and Y. Qiao, "EdgePlus: A multiagent reinforcement learning framework for dynamic task allocation in 6G edge computing," IEEE Internet of Things Journal, vol. 13, no. 8, pp. 15 514–15 532, 2026.

[2] J. F. C. Kingman, "The single server queue in heavy traffic," Mathematical Proceedings of the Cambridge Philosophical Society, vol. 57, no. 4, pp. 902–904, 1961.

[3] B. K. Chaurasia, M. M. Shukla, V. Vishwakarma, and B. Tiwari, "Transformative impact of edge computing with 6G network using cloud computing," Journal of Cloud Computing, 2026.

[4] S. Chauhan, Z. B. Patel, S. Ranu, R. Sen, and N. Batra, "AirDelhi: Fine-grained spatio-temporal particulate matter dataset from Delhi for ML based modeling," in Advances in Neural Information Processing Systems (Datasets and Benchmarks Track), vol. 36, 2023, pp. 75 455–75 468.

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, 1973.

[6] J. R. Jackson, "Scheduling a production line to minimize maximum tardiness," Management Science Research Project, University of California, Los Angeles, Research Report 43, 1955.

[7] C. D. Locke, "Best-effort decision making for real-time scheduling," Ph.D. dissertation, Carnegie Mellon University, 1986.

[8] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. Springer, 2011.

[9] M. Weiser, B. Welch, A. Demers, and S. Shenker, "Scheduling for reduced CPU energy," in Proc. 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI), 1994.

[10] P. Pillai and K. G. Shin, "Real-time dynamic voltage scaling for low-power embedded operating systems," in Proc. 18th ACM Symposium on Operating Systems Principles (SOSP), 2001, pp. 89–102.

[11] 3GPP, "Evolved universal terrestrial radio access (E-UTRA); further advancements for E-UTRA physical layer aspects," 3rd Generation Partnership Project, Technical Report TR 36.814, 2010.

[12] CityPulse Project, "CityPulse smart city datasets: Pollution measurements," University of Surrey, http://iot.ee.surrey.ac.uk:8080/datasets.html, accessed Nov. 2026.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[14] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in Proc. International Conference on Learning Representations (ICLR), 2016.

[15] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[16] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," Journal of the Royal Statistical Society: Series B, vol. 57, no. 1, pp. 289–300, 1995.

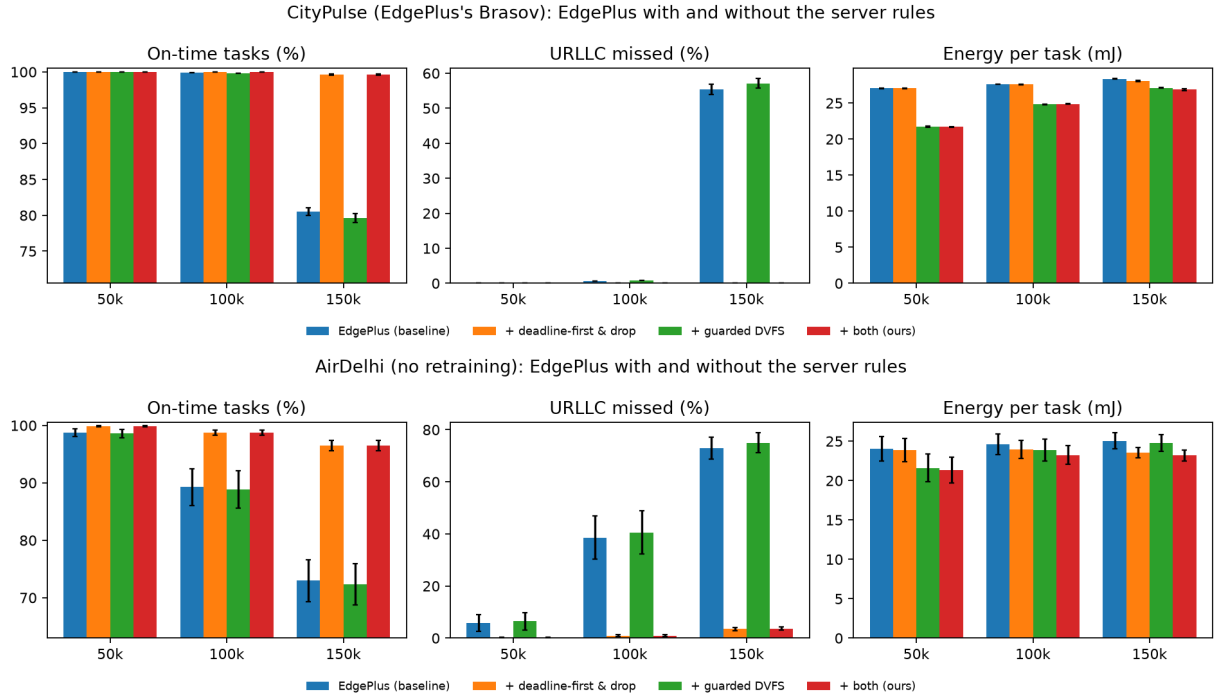



Fig. 1. On-time completion, URLLC misses and energy per task for CityPulse (top) and AirDelhi without retraining (bottom). Error bars are 95% confidence intervals over 30 paired runs.

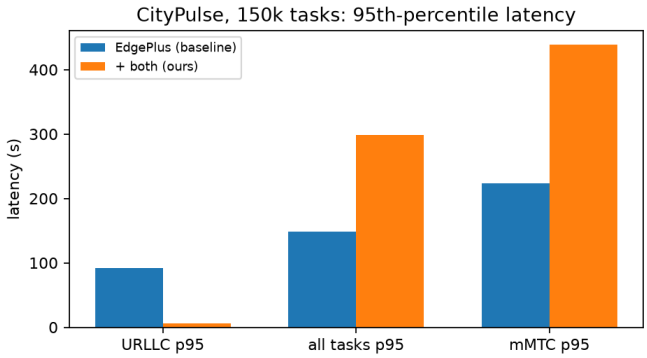



Fig. 2. 95th-percentile latency by class, CityPulse at 150,000 tasks. URLLC tasks finish much sooner; mMTC tasks wait longer but stay within their 120–600 s deadlines.